

NAZIV

System V ABI AMD64 – Zadaci za vezbu

1. DATA REPRESENTATION (3.1.2)**Zadatak 1 – Velicine tipova**

Za svaki C tip navedi: (a) velicinu u bajtovima (LP64), (b) alignment:

1. char 2. short 3. int 4. long (LP64)
5. long long 6. void* 7. double 8. __int128

Zadatak 2 – Strukture i poravnanje

```
struct S {
    char a;
    int b;
    char c;
    long d;
    short e;
};
```

- (a) Koji je alignment cele strukture?
(b) Na kojim su offset–ima svaki clan (uz padding)?
(c) Kolika je ukupna velicina?
(d) Kako reorganizovati clanove da minimizujes velicinu?

Zadatak 3 – Bit–fields

```
struct Flags {
    unsigned int ready : 1;
    unsigned int mode : 3;
    unsigned int count : 12;
    unsigned int unused : 16;
};
```

- (a) U kom smeru se alociraju bit–fields (MSB ili LSB prvo)?
(b) Kolika je velicina ove strukture?
(c) Da je mode=5 i count=0xABC, kako izgleda sadrzaj u memoriji (hex)?

Zadatak 4 – Boolean i pokazivaci

- (a) Funkcija vraca bool. Koja je vrednost %rax ako vraca true?
Sta ako %rax = 0xFF – true ili false?
(b) Koji je hex–zapis null pokazivaca?
(c) __int128 vrednost 0x0102...0F10 cuva se u memoriji.
Navedi redosled bajtova na adresama [0x1000..0x100F].

2. REGISTRI I CALLING CONVENTION (3.2.1)**Zadatak 5 – Klasifikacija registara**

Razvrstaj u Caller–saved i Callee–saved:

rax, rbx, rcx, rdx, rsi, rdi, rbp, rsp, r8, r9, r10, r11, r12, r13, r14, r15

Zadatak 6 – Sta treba sacuvati?

Funkcija foo koristi: rax, rbx, r12, r13, rdi.

- (a) Koje registre foo mora da push–uje pre upotrebe?
(b) Koje sme slobodno da koristi bez cuvanja?
(c) Ako foo poziva bar, koje vrednosti moze da izgubi?

Zadatak 7 – Analiza prologue

```
push rbp
push rbx
push r12
```

```
mov  rbp, rsp
sub  rsp, 32
```

- (a) Koje registre ova funkcija cuva i zasto?
- (b) Koliko je ukupno bajtova rezervisano na steku (ukljucujuci push-ove)?
- (c) Da li je %rsp poravnat na 16 bajtova?

3. STACK FRAME (3.2.2)

Zadatak 8 – Stack layout

main poziva func(int a, int b). Pre CALL: rsp = 0x7FFFFFFF0.

- (a) Sta se desi sa rsp nakon CALL?
- (b) Sta cuva [rsp] po ulasku u func?
- (c) Func: push rbp; mov rbp, rsp. Koji je rbp?
- (d) sub rsp, 32. Da li je stek poravnat?

Zadatak 9 – Red Zone

- (a) Sta je red zone i koliko je velika?
- (b) Koja vrsta funkcije sme da je koristi?
- (c) Sta se desi sa red zone ako funkcija pozove drugu funkciju?
- (d) Moze li signal handler pokvariti red zone?

Zadatak 10 – Stack poravnanje

Pravilo: (rsp+8) je uvek visekratnik od 16 pri ulasku u funkciju.

- (a) Zasto je uslov (rsp+8) a ne samo rsp?
- (b) Pozivas funkciju sa __m256 argumentom. Koji alignment se zahteva?
- (c) Da li ce stek biti poravnat pri pozivu foo()?


```
sub rsp, 8
call foo
```

4. PARAMETER PASSING (3.2.3)

Zadatak 11 – Klasifikacija argumenata

Klasifikuj (INTEGER, SSE ili MEMORY):

1. int 2. double 3. char* 4. long long 5. float
6. struct { int a,b,c,d; } 7. struct { long a[5]; } 8. __int128

Zadatak 12 – Koja C funkcija?

- (a)


```
mov  edi, 42
mov  esi, 7
call func_a ; vraca u rax
```
- (b)


```
movsd xmm0, [some_double]
mov  edi, 1
call func_b ; vraca u xmm0
```
- (c)


```
mov rdi, rax ; pokazivac
mov rsi, 100 ; velicina
mov rdx, 0   ; flags
call func_c
```

Zadatak 13 – Pisanje prologue/epilogue

Napisi prologue i epilogue za:

```
long compute(long x, long y, long z);
// lokalne: long a, b (16B); koristi rbx interno
```

5. KOMBINOVANI ZADACI

Zadatak 14 – Analiza koda

```
void mystery(int *arr, int n) {  
    long tmp;  
    int i;  
    char flag;  
}
```

- (a) Koji registri nose arr i n?
- (b) Koliko prostora zauzimaju lokalne varijable uz padding?
- (c) Može li koristiti red zone ako ne poziva drugu funkciju?

Zadatak 15 – Debugiranje ABI greske

```
my_func:  
    mov r12, rdi  
    call some_other_func  
    mov rax, r12  
    ret
```

Pronadji ABI gresku. Hint: razmisli o rsp poravnanju pri CALL.

ODGOVORI – KRATKI KLJUC

(Pokusaj sam pre gledanja!)

Zadatak 1: char=1/1, short=2/2, int=4/4, long=8/8, long long=8/8, void*=8/8, double=8/8, __int128=16/16

Zadatak 5 – Caller–saved: rax, rcx, rdx, rsi, rdi, r8–r11

Callee–saved: rbx, rbp, r12–r15, rsp

Zadatak 9a: Red zone = 128 bajtova ispod rsp, netaknuta od signal handlera.

Zadatak 15: my_func ne poravnava rsp pre CALL. Treba **sub rsp, 8** pre call (ili push nekog callee–saved registra).